

PHINMA-COC

Carmen, Cagayan de Oro City

OFFICIAL FINAL PROJECT TECHNICAL DOCUMENTATION

Computer Shop Management System

Technical System Manual, Deployment Guide & Defense Framework

Course: Information Management 2 (BSIT)

Submitted by:

RONALD ALLAN ZAMORA

BHONG S. KILATON

ANSARY M. ASIS

AL KENNETH M. UNTALAN

RYAN V. GESTA

KARL JOROSH C. PACULAN

Section: **COC-FAB-BSIT2-14**

Date: 10/13/26

1. System Overview

The Computer Shop Management System replaces the coin-operated pisonet timer box with a counter-controlled rental system. Staff start, extend, and stop PC sessions from a central dashboard, each customer PC shows a live countdown with automatic reminders, billing is computed from hours multiplied by the hourly rate, and the counter can broadcast notices to every active PC.

Tech stack: Java Swing frontend (NetBeans Design view with hand-written runtime code), JDBC backend running on the Java JDK (NetBeans default platform; project source/target level 26 per nbproject/project.properties), MySQL/MariaDB via the XAMPP local server, mysql-connector-j-9.4.0.

Key features:

- Role-based login separating counter staff from customer PCs.
- Live counter dashboard with one card per PC, refreshed every second.
- Session Start, Extend, Stop, and one-step Undo of the last extend.
- Customer countdown with 30-minute, 5-minute, and time's-up popups.
- Automatic expiry of finished sessions with no click required.
- Counter-to-PC broadcast messaging with per-PC delivery tracking.

Both windows poll the shared database once per second, so the counter and every customer PC stay in sync within about a second with no networking code. Out of scope by design: printed reports, customer membership accounts, receipt printing, remote control of customer PCs, and multi-branch support. Sales totals are computed for the current day only; history remains queryable in the tables.

2. Database Design & Schema Normalization (3NF)

Database name: java_database (InnoDB engine on all tables).

2.1 Schema Entities (5 Tables)

login_table — Counter and customer login accounts.

Column	Type	Constraints	Purpose
id	INT AUTO_INCREMENT	PK	Row identifier
username	VARCHAR(50)	UNIQUE NOT NULL	Login name
password	VARCHAR(100) NOT NULL	—	SHA-256 hex digest (64 chars), never plaintext
role	VARCHAR(20) NOT NULL	DEFAULT 'counter'	counter opens the dashboard, customer opens the PC timer

Stores the four seed accounts (admin + pc1–pc3). Passwords are hashed by the application before comparison. The table has no foreign keys by design: authentication must work even if every other table is empty, and no other table ever points back at an account.

pcs — The shop's physical computers.

Column	Type	Constraints	Purpose
id	INT AUTO_INCREMENT	PK	Row identifier
pc_name	VARCHAR(20)	UNIQUE NOT NULL	PC1, PC2, PC3
hourly_rate	DECIMAL(10,2) NOT NULL	DEFAULT 25.00	Reserved per-PC rate (running code applies one shared P25.00 constant)
broadcast_pending	TINYINT(1) NOT NULL	DEFAULT 0	1 = a broadcast is waiting for this PC
broadcast_message	VARCHAR(255) NOT NULL	DEFAULT ''	The waiting broadcast text

Stores one row per machine. Broadcast columns carry transient per-PC state, never history. The counter dashboard reads this table once at startup to learn PC ids, then watches sessions for live state.

sessions — One row per rental session, written only by the counter.

Column	Type	Constraints	Purpose
id	INT AUTO_INCREMENT	PK	Row identifier
pc_id	INT NOT NULL	FK → pcs.id	Which PC this session ran on
customer_id	INT	NULLABLE FK → customers.customer_id	Who rented (−1/NULL = walk-in, no record)
hours	INT NOT NULL	—	Purchased hours (1–6)
amount	DECIMAL(10,2) NOT NULL	—	Billed cost, frozen at sale time
start_time	DATETIME NOT NULL	—	Session start (database clock)
end_time	DATETIME NOT NULL	—	Session end (database clock)
status	VARCHAR(20) NOT NULL	DEFAULT 'active'	active or done
is_archived	TINYINT NOT NULL	DEFAULT 0	0 = live, 1 = archived

Stores the complete rental history. Rows are never deleted. The daily sales total sums this table's amount for rows started today, so yesterday's business never leaks into today's figure while remaining fully queryable.

customers — Walk-in customers the counter chooses to record.

Column	Type	Constraints	Purpose
customer_id	INT AUTO_INCREMENT	PK	Row identifier
customer_name	VARCHAR(100) NOT NULL	—	Customer name
contact_number	VARCHAR(20)	NULLABLE	Phone number (optional)
email	VARCHAR(100)	UNIQUE NULLABLE	Deduplication key (optional)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Record creation time

Stores one row per known customer. The application reuses an existing row when a typed email matches, and skips the table entirely when no name is given. Sessions left by anonymous walk-ins carry a NULL customer_id instead of a fabricated customer row, keeping the table free of junk records.

Seed rows shipped with the setup script (passwords stored hashed in the real table):

```
INSERT INTO login_table (username, password, role) VALUES
 ('admin', '<sha256-of-12345>', 'counter'),
 ('pc1', '<sha256-of-12345>', 'customer'),
 ('pc2', '<sha256-of-12345>', 'customer'),
 ('pc3', '<sha256-of-12345>', 'customer');
INSERT INTO pcs (pc_name, hourly_rate) VALUES
 ('PC1', 25.00), ('PC2', 25.00), ('PC3', 25.00);
```

Foreign-key enforcement is active on all tables (InnoDB engine): inserting a session for a nonexistent PC, or a log entry for a nonexistent session, is rejected by MySQL with a constraint error, which the application surfaces as a friendly dialog.

session_log — Append-only audit trail of session state changes.

Column	Type	Constraints	Purpose
log_id	INT AUTO_INCREMENT	PK	Row identifier
session_id	INT NOT NULL	FK → sessions.id	Which session changed

action	VARCHAR(30) NOT NULL	—	START, EXTEND, STOP, or UNDO_EXTEND
details	VARCHAR(255)	NULLABLE	Human-readable detail (e.g. "Extended by 2 hour(s)")
log_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the change happened

Stores one row per state change, newest first for the log viewer. Never updated or deleted. Because every write carries its session id, a single WHERE session_id = ? query reconstructs the full life story of any session for dispute resolution.

2.2 Normalization Breakdown (1NF → 3NF)

- **1NF:** Every column holds one atomic value — no lists, no comma-separated data in any cell. Every table has a designated primary key and no repeating groups exist. Concrete check: sessions stores one fact per column per session (hours in hours, cost in amount); at no point does any column pack two facts together.
- **2NF:** Every non-key attribute depends on the full primary key. All five primary keys are single-column surrogate ids, so partial dependence on part of a key is structurally impossible: each session's hours, amount, times, status, and flags describe that whole session row and nothing else.
- **3NF:** No transitive dependencies — no non-key attribute depends on another non-key attribute. The hourly price lives exactly once per PC in pcs.hourly_rate instead of being copied into every sessions row. The per-session amount looks like duplication but is not a dependency violation: it is the frozen historical charge (hours × rate at sale time), so old bills stay correct even if the rate ever changes. Customer names live only in customers.customer_name; sessions reference them by id.

2.3 Crow's Foot ERD

Three one-to-many relationships and one standalone entity:

- pcs (1) ———< sessions (M): one PC accumulates many session rows over time; each session belongs to exactly one PC (sessions.pc_id). Deleting a PC with history is rejected by the foreign key, which is the desired protection.
- customers (1) ———< sessions (M): one recorded customer can rent many times; each session links to at most one customer (sessions.customer_id, nullable for anonymous walk-ins, so the relationship is optional on the session side).

- sessions (1) ———< session_log (M): one session produces many log entries; each entry belongs to exactly one session (session_log.session_id). Logs are append-only, so this relationship only ever grows.
- login_table: standalone, no foreign keys; governs login only.

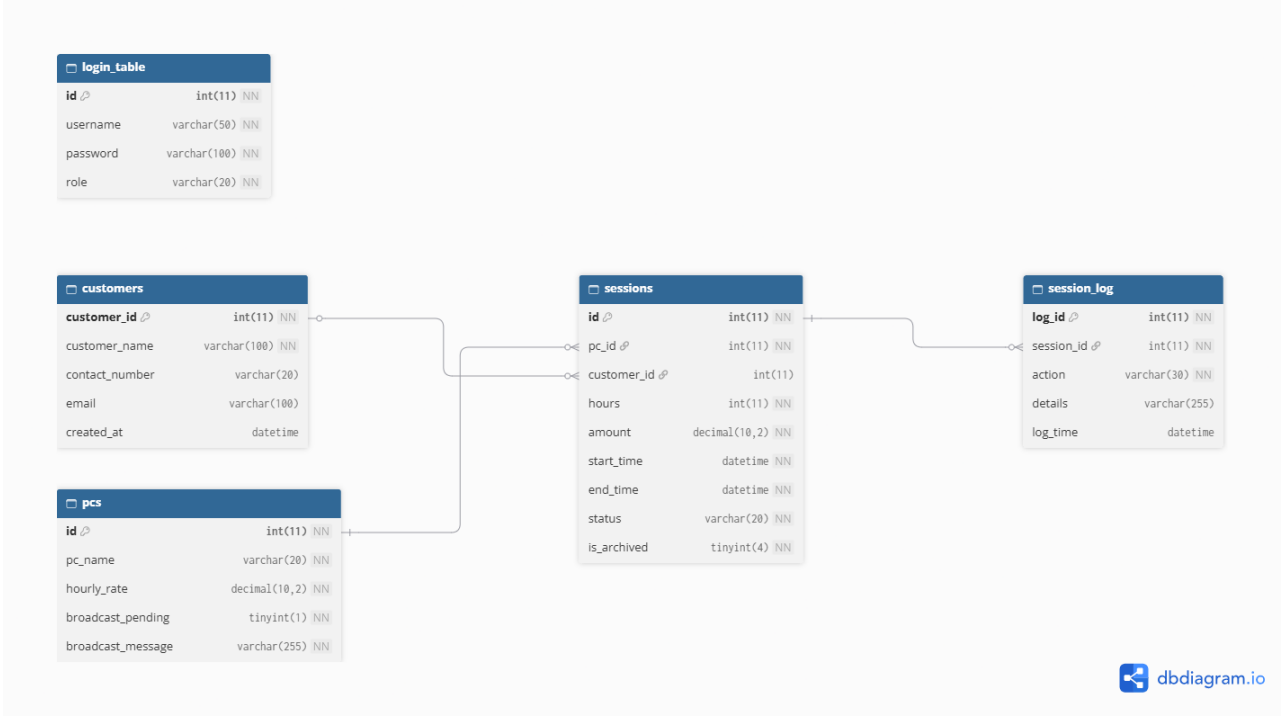


Figure 1. Crow's Foot ERD — Computer Shop Management System schema (PK/FK, cardinalities, and session relationships).

3. Logic Specifications (Step-by-Step Data Flow)

Each feature below names the rubric criterion it demonstrates:

Feature	Proves
Starting a Session	CREATE (new row + generated key)
Extending a Session	UPDATE (same row, pushed forward)
Undo Last Extend	UPDATE + Stack discipline (exact reversal)
Archiving an Expired Session	ARCHIVE via soft delete (row preserved)

3.1 Feature: Starting a Session

1. Staff clicks Start on a PC card (refused at once if the PC is already in use).
2. A popup asks for the customer name (optional); if a name is typed, a second popup asks for email (optional).

3. SessionDAO.registerOrGetCustomer() looks up the email in customers and returns the existing id on a match, otherwise inserts a new row and returns its generated id. Deduplication keys on email only when one was typed; nameless sessions skip the lookup entirely and proceed with customer id −1 (the application's "no customer" marker, stored as NULL).
4. SessionDAO.startSession() inserts the session row with start_time = NOW() and end_time = DATE_ADD(NOW(), INTERVAL ? HOUR):

```
INSERT INTO sessions (pc_id, customer_id, hours, amount, start_time, end_time, status, is_archived)
VALUES (?, ?, ?, ?, NOW(), DATE_ADD(NOW(), INTERVAL ? HOUR), 'active', 0)
```

5. The generated session id is read back and logEvent writes a START entry to session_log.
6. The dashboard repaints on the next tick, showing the customer name on the card.

Validation and errors: empty login input never reaches the database; a dead connection produces "Cannot connect to database. Please start XAMPP." instead of a stack trace; wrong credentials produce "Invalid username or password!".

3.2 Feature: Extending a Session

1. Staff clicks Extend on an in-use PC (refused with a message if no session is active).
2. dsm.pushExtension(sessionId, hours, amount) records the exact extended values on the Stack.
3. SessionDAO.extendSession() runs:

```
UPDATE sessions SET end_time = DATE_ADD(end_time, INTERVAL ? HOUR), amount = amount + ?, hours = hours + ? WHERE id = ?
```

4. logEvent writes an EXTEND entry recording how many hours were added.

Validation and errors: Extend on a PC with no active session is refused before any SQL runs ("has no active session"); database failures surface the database message in a dialog and the screen repaints to the true state on the next tick.

3.3 Feature: Undo Last Extend (Stack LIFO)

1. Staff clicks Undo Last Extend (refused with "Nothing to undo." when the Stack is empty).
2. dsm.undoLastExtension() pops the most recent ExtendRecord — last in, first out, carrying the exact hours and amount that extend added.
3. SessionDAO.revertLastExtend() subtracts precisely those values:

```
UPDATE sessions SET end_time = DATE_SUB(end_time, INTERVAL ? HOUR), hours = hours - ?, amount = amount - ? WHERE id = ?
```

4. logEvent writes an UNDO_EXTEND entry, so the audit trail shows both the extend and its reversal.

Validation and errors: Undo with an empty Stack is refused ("Nothing to undo."); the reversal subtracts exactly the hours and amount the popped record added, so repeated undos walk the session back step by step without ever overshooting into negative values on a correctly extended session.

3.4 Feature: Archiving an Expired Session (Soft Deletion)

1. The dashboard's javax.swing.Timer fires every second.
2. SessionDAO.autoStopExpired() closes every session whose end time has passed:

```
UPDATE sessions SET status = 'done', is_archived = 1 WHERE status = 'active' AND is_archived = 0 AND end_time <= NOW()
```

3. Rows are preserved for audit (never hard-deleted); finished sessions simply stop matching the active-session queries, so PCs return to AVAILABLE on their own. Bulk expiry is deliberately not logged per row; the method carries a comment stating a production system would use a trigger or scheduled job instead.

Validation and errors: the bulk statement affects zero rows when nothing has expired, which is the normal case and needs no feedback; genuine failures print to the console only, by design, so the operator is never flooded with one popup per second.

4. System Deployment & Setup Guide (Imperative Mood)

Step 1: Open the XAMPP Control Panel and start Apache and MySQL. Both modules must show green "Running" status before anything else works.

Step 2: Open a browser and go to http://localhost/phpmyadmin/ — the database console should load with no login prompt on a default install.

Step 3: Create the database java_database. It appears empty in the left sidebar; that is expected until Step 4.

Step 4: Import setup.sql (creates login_table, pcs, sessions, customers, session_log with keys, plus seed data). A success banner confirms all statements ran; any red error means Step 3 was skipped.

Step 5: Open NetBeans and load the ComputerShopManagement project. It must open with zero error badges in the Projects pane.

Step 6: Verify mysql-connector-j-9.4.0.jar is listed under Libraries. Without it, compilation fails on the very first JDBC import.

Step 7: Run the project. Log in with the default credentials below. (Type the plaintext password; the application hashes it with SHA-256 before comparing to the stored digest.)

Verify the installation with these read-only checks in phpMyAdmin (all must return rows, none modify data):

```
SHOW TABLES FROM java_database;
SELECT COUNT(*) FROM login_table;
SELECT COUNT(*) FROM pcs;
SELECT COUNT(*) FROM sessions;
```

Expected: 5 tables present; login_table holds 4 accounts; pcs holds PC1–PC3. If the application shows "Cannot connect to database," XAMPP's MySQL module is not running — start it and retry; no reinstall is needed.

5. Core Competencies & Security Matrix

- **SQL Injection Prevention:** all queries use PreparedStatement with ? placeholders — every value from the screen (usernames, hours, messages, names, emails) is bound as data, so typed input can never execute as SQL code. Applies across all data-access classes (connectDB, LoginDAO, SessionDAO, PCsDAO); zero string-concatenated queries exist in the codebase.
- **Password Security:** passwords are stored and compared as SHA-256 hex digests via PasswordUtil (64 characters); plaintext never reaches the database.
- **Referential Integrity:** foreign keys on sessions.pc_id, sessions.customer_id, and session_log.session_id reject orphan rows, and UNIQUE on usernames, PC names, and emails rejects duplicates.
- **Soft Deletion:** ending a session flips status/is_archived flags instead of deleting; no DELETE statement exists in the codebase, so the full audit trail survives.
- **Connection Handling:** try-with-resources closes every connection, statement, and result set; a dead database produces a friendly dialog, never a crash or a leaked connection.
- **Input Validation:** trim() on all user input before binding; empty login input and empty broadcasts are rejected before any database call runs.
- **Audit Trail:** session_log records every state change (START, EXTEND, STOP, UNDO_EXTEND) with session id, detail text, and timestamp, viewable from the dashboard's View Logs button (SessionDAO.getRecentLogs, newest 20).

6. Test Credentials

Username	Password	Role	Purpose
admin	12345	counter	Full Dashboard access
pc1	12345	customer	Customer timer for PC1
pc2	12345	customer	Customer timer for PC2

pc3	12345	customer	Customer timer for PC3

7. Final Deliverable Assembly & Workspace Checklist

Project Application Name: Computer Shop Management System

Course & Section: Information Management 2 (BSIT)

Step 1: UI Consistency & Source Code Hygiene

- Unused testing variables, hardcoded dummy values, and debugging print statements removed.
- Fonts, margins, and button formatting match seamlessly across all interface screens.
- Raw database errors replaced with custom feedback validation banners.

Step 2: Database Schema Script Generation

- All table creations, foreign key constraints, and seed data exported into a single .sql text file.
- Schema script organized in chronological execution order for error-free deployment.
- Personal test data removed; seeded with standard academic test records.

Master Schema File Name: setup.sql

Step 3: Document Package Assembly

- Setup guide written strictly in step-by-step imperative mood with default credentials.
- Master documentation cleared of boilerplate placeholder text or empty headings.
- Documentation file tree mirrors the active repository directory structure.
- Crow's Foot ERD diagram generated from the actual schema and embedded in Section 2.3.

Master Technical Manual File Name: TECHNICAL_DOCUMENTATION.md

STUDENT CONFIRMATION: I verify that the code repository, database schema, and documentation assets checked above match our final production submission package.

Student Signature: _____ Date: _____